

Réalisation Système

TP2 : Plante Connectée à la base de la carte Nucleo-WB55RG

Discipline : Master 1 - SME

Réalisé par :

NASSIME BOUANANI
KAMDA TEZEBO DIBREY JONATAN

Assisté par :

Pr Thierry Perisse

Année universitaire : 2025/2026

Table des matières

Objectif :	2
Procédures et progression :	2
Idée générale sur le projet :	2
Caractéristiques de la carte Nucleo-WB55RG :	2
Manipulation 1 : Allumer la LED avec Bluetooth	3
Manipulation 2 : Envoyer des données de capteurs par Bluetooth	5
Manipulation 3 : Programmation et test de capteurs d'humidité et de la lumière + Affichage de ces données sur LCD	5
Caractéristiques du capteur d'humidité :	6
Caractéristiques du capteur de la lumière :	6
Manipulation 4 : Programmation et test du MOSFET	9
Manipulation 5 : Test de l'ensemble de composants	10
Manipulation 6 : Commande du système par téléphone (Bluetooth)	10
Manipulation 7 : Rendre le système autonome (alimentation externe)	12
Bibliographie :	13

Listes de figures

Figure 1: Le schéma général du système [1]	2
Figure 2: Caractéristiques et architecture de la carte STM32 [2]	3
Figure 3: Emplacement du projet	4
Figure 4: Les lignes de codes rajoutées avec leurs emplacements	5
Figure 5 : Les données reçues et affichées sur l'application	5
Figure 6 : Capteur d'humidité [4]	6
Figure 7: Capteur de la lumière [5]	6
Figure 8 : Affichage de données de capteurs humidité et luminosité sur LCD 16x02	8
Figure 9: Affichage de données de capteurs humidité et luminosité sur LCD	10
Figure 10 : Affichage de données de capteurs humidité et luminosité sur LCD	10
Figure 11: Captures d'écran de l'application Bluetooth	11
Figure 12: Rapport cyclique défini dans le code 10% et 20%	12
Figure 13: Système Complet testé en autonomie	12
Figure 14 : Brochage de Vbat [3]	13
Figure 15 : Analyseur de spectre de la fréquence BLE	13

Objectif :

Le kit plante connectée comprend tous les éléments nécessaires pour réaliser un montage permettant de :

- Automatiser l'arrosage d'une plante en fonction du taux d'humidité du sol détecté par un capteur.
- Apprendre les bases de l'électronique et de la programmation en manipulant des capteurs et des actionneurs.
- Optimiser la consommation d'eau en apportant à la plante uniquement la quantité nécessaire, sans intervention humaine.
- Assurer la survie des végétaux en toute autonomie, idéalement lors d'absences prolongées ou de vacances.

Procédures et progression :

Dans un premier temps, nous avons pris en main la carte Nucleo-WB55RG. Cette phase d'initiation a permis de configurer l'environnement et de réaliser des fonctions de base, telles que le pilotage de LED et la communication de données via Bluetooth (BLE).

Par la suite, nous avons intégré différents composants matériels : l'acquisition de données via les capteurs de luminosité et d'humidité Grove, ainsi que le contrôle d'une pompe hydraulique par un module MOSFET, piloté en PWM pour gérer ses cycles d'activation.

Idée générale sur le projet :

Le kit plante connectée comprend les éléments pour réaliser le suivi de l'humidité du sol et de la luminosité et le contrôle de l'arrosage automatique d'une plante.

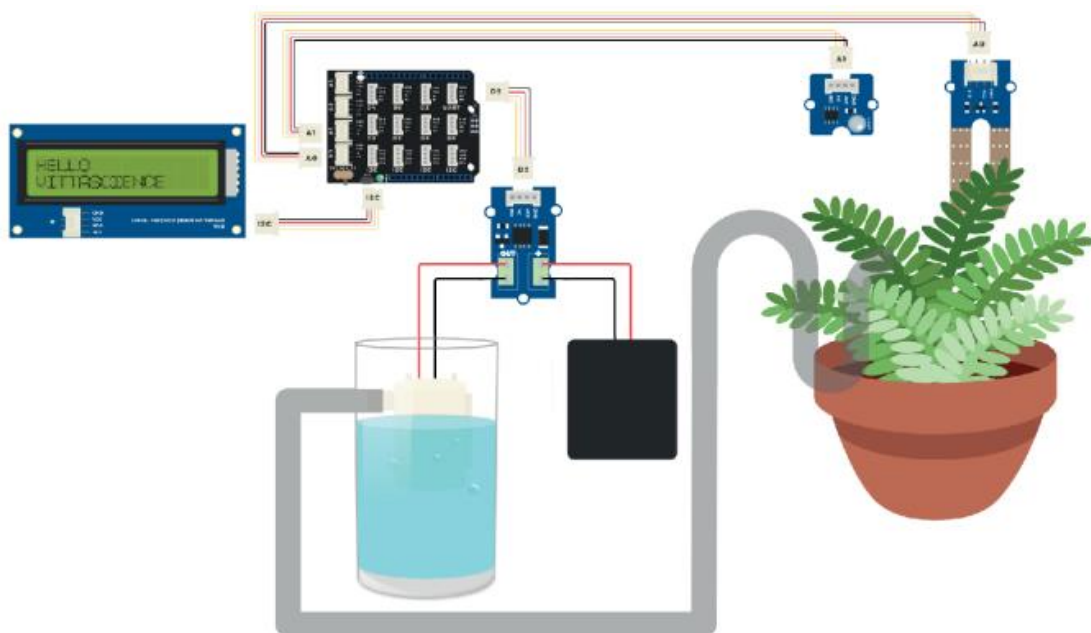


Figure 1: Le schéma général du système [1]

Caractéristiques de la carte Nucleo-WB55RG :

La carte NUCLEO-WB55RG est une plateforme de développement particulièrement intéressante car elle repose sur une architecture dual-core (double cœur) conçue spécifiquement pour les applications sans fil (Bluetooth, Zigbee, Thread). [2]

Caractéristique	Cortex-M4 (Principal)	Cortex-M0+ (Coprocesseur)
Rôle	Gère votre code applicatif (capteurs, calculs, écran).	Dédié exclusivement à la pile radio (Stack BLE, Zigbee).
Fréquence	Jusqu'à 64 MHz	32 MHz
Particularité	Possède une unité de calcul flottant (FPU) et instructions DSP .	Sécurisé et inaccessible directement par l'utilisateur.
Avantage	Votre application ne ralentit pas quand le Bluetooth communique.	Garantit que la connexion sans fil reste stable et synchrone.

Diagramme blocs du SoC STM32WB55RG

Architecture du STM32WB55RG

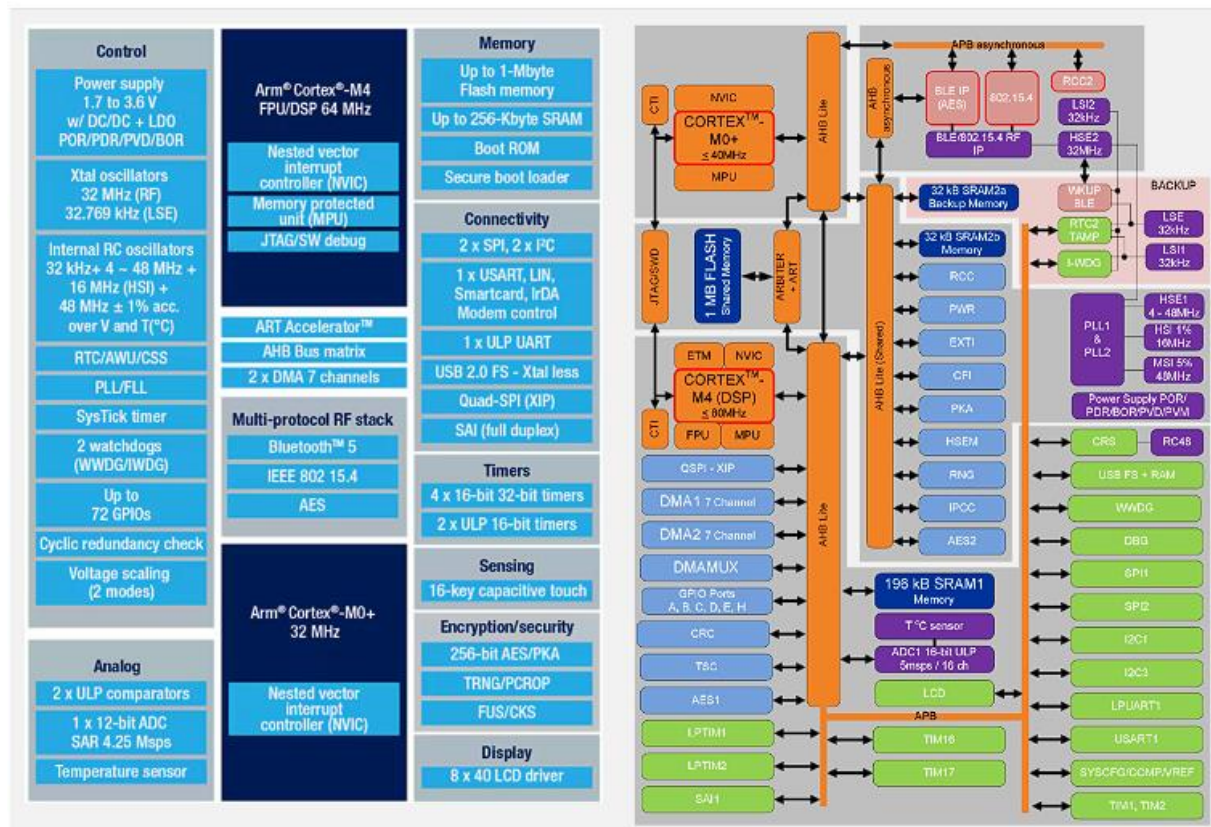


Figure 2: Caractéristiques et architecture de la carte STM32 [2]

Manipulation 1 : Allumer la LED avec Bluetooth

On vous met le lien de la vidéo qui nous a permis de configurer et programmer le BLE de la carte en utilisant le logiciel STM32CubeIDE :

<https://www.youtube.com/watch?v=CKYWy7LKL68&list=PLFPGZOSELXrUXxjXOadHUcZmfd1o9e3mp&index=4>

On a rajouté cette ligne de commande dans le fichier custom_stm.c, et qui nous a permis de commuter entre les états du LED quand la carte reçoit une information du BLE de téléphone

HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);

```
144     else if(attribute_modified->Attr_Handle == (CustomContext.CustomMycharwriteHdle + CHARACTERISTIC_VALUE_ATTRIBUTE_OFFSET))
145     {
146         return_value = SVCCTL_EvtAckFlowEnable;
147         /* USER CODE BEGIN CUSTOM_STM_Service_1_Char_1_ACI_GATT_ATTRIBUTE_MODIFIED_VSEVT_CODE */
148         HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
```

Figure : Code a rajouté

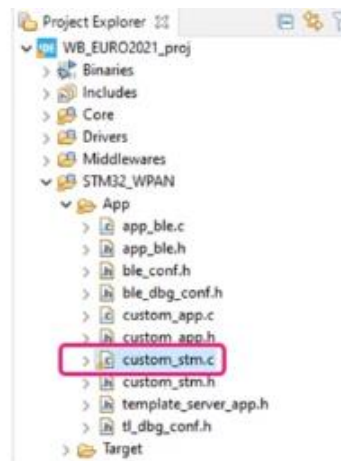
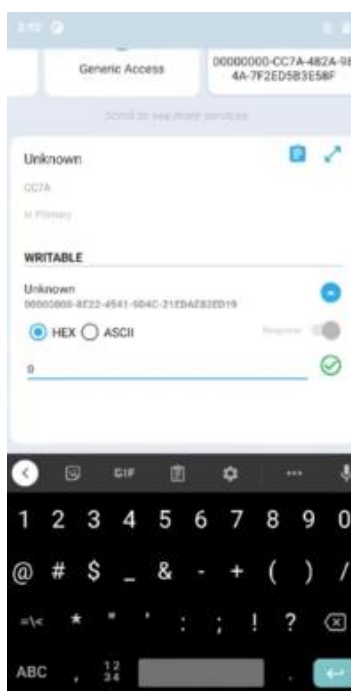


Figure 3: Emplacement du projet



LED allumée en envoyant d'ordre depuis le BLE du téléphone

Figure : Code a rajouté

Manipulation 2 : Envoyer des données de capteurs par Bluetooth

On vous met le lien de la vidéo qui nous a permis de configurer et programmer le BLE de la carte en utilisant le logiciel STM32CubeIDE :

<https://www.youtube.com/watch?v=Zgw3wRpGSRQ>

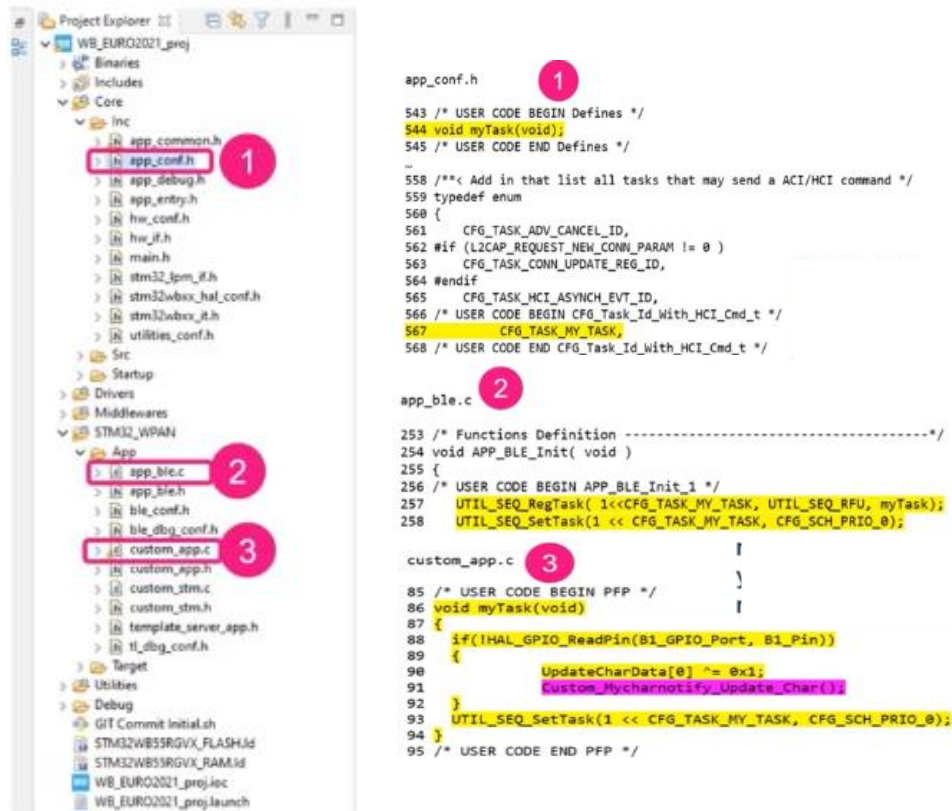


Figure 4: Les lignes de codes rajoutées avec leurs emplacements

Comme vous voyez à côté, les données des capteurs d'humidité et de la lumière sont émises par le BLE de la carte wb55rg et reçues par le BLE du téléphone.

Cependant, elles sont affichées en ASCII et par la suite on a travaillé sur un code python qui permet de convertir ces données en valeur décimale et puis l'afficher en temps réel sous forme de graphe.

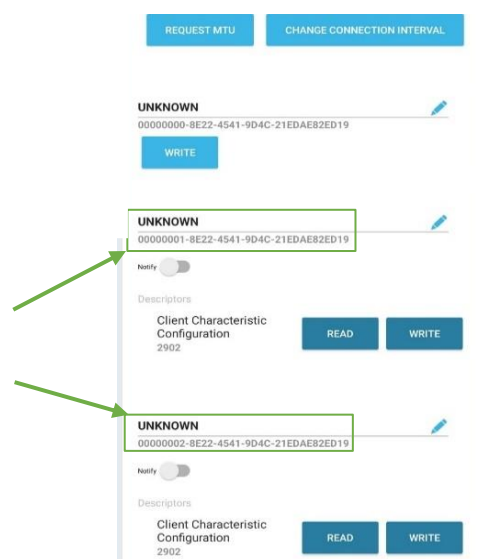


Figure 5 : Les données reçues et affichées sur l'application

Manipulation 3 : Programmation et test de capteurs d'humidité et de la lumière + Affichage de ces données sur LCD

Caractéristiques du capteur d'humidité :

Selon le site seed studio : [4]

Voltage = 5 V

Current = 35 mA

Output Value

- Condition: Sensor in dry soil (from 0 to 300)
- Condition: Sensor in humid soil (from 300 700)
- Condition: Sensor in water (from 700 to 950)

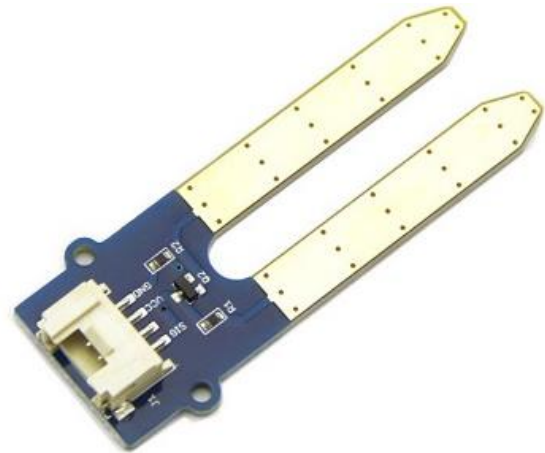


Figure 6 : Capteur d'humidité [4]

5V	Rouge
GND	Noire
NC	Blanc
SIG - Brochage	Jaune - A0

Caractéristiques du capteur de la lumière :

Selon le site seed studio : [5]

Operating voltage = 3~5V

Operating current = 0.5~3 mA

Response time= 20-30 milliseconds

Peak Wavelength = 540 nm

Weight = 4 g



Figure 7: Capteur de la lumière [5]

5V	Rouge
GND	Noire
NC	Blanc
SIG - Brochage	Jaune – A1

Code rajouté dans le main.c concernant du capteur humidité :

```
// Variables
uint16_t moisture=0;
uint16_t raw_data[2];
// Callback de conversion ADC
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc) {
    convCompleted = 1; }

// Dans la boucle while(1)
if (convCompleted) {
    moisture = raw_data[0]; // Lecture de l'humidité

    // Gestion de l'arrosage automatique
    if (moisture >= 0 && moisture < 300) {
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 400); // 40% pour sol très sec
    }
    else if (moisture >= 300 && moisture < 700) {
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 200); // 20% pour sol peu humide
    }
    else if (moisture >= 700) {
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 0); // Arrêt si sol humide
    }

    // Affichage du pourcentage
    aff_mois = (prev_mois * 100) / MAX_MOISTURE; // MAX_MOISTURE = 2500
    set_mois(moisture);
}
```

Code rajouté dans le main.c concernant du capteur de lumière :

```
// Variables
uint16_t lux = 0;
uint16_t raw_data[2];

// Dans la boucle while(1)
if (convCompleted) {
    lux = raw_data[1]; // Lecture de la luminosité

    // Affichage du pourcentage
    aff_lux = (prev_lux * 100) / MAX_LUX; // MAX_LUX = 3000

    // Mise à jour dans la stack Bluetooth
    set_lux(lux);
}
```

Après avoir programmé ces deux capteurs, et selon la configuration sur le logiciel STM32CubeIDE, on a branché le capteur d'humidité sur le port A0 du Shield, le capteur de la lumière sur le port A1 et LCD sur le port d'I2C. Voici le résultat du test effectué sur cette partie.

Interprétation : 5 pourcents d'humidité puisque les deux barres en cuivre du capteur sont secs et la lumière vaut 281 qui représente la quantité de la lumière reçue.

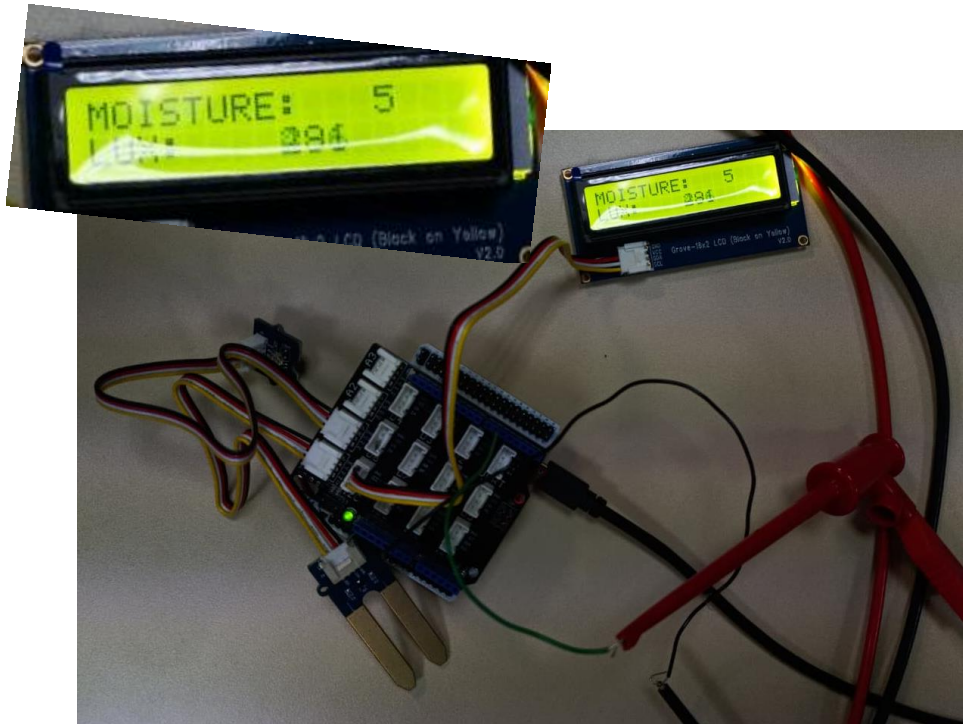


Figure 8 : Affichage de données de capteurs humidité et luminosité sur LCD 16x02

Pour valider le bon fonctionnement de notre système, nous avons procédé à un test de mesure directe à l'aide d'un multimètre. En plongeant les deux barres de cuivre du capteur d'humidité Grove dans l'eau, nous avons pu vérifier sa réactivité et établir une valeur de référence. Cette valeur nous servira de base de comparaison pour décider, par la suite, du déclenchement ou non de l'irrigation.

Sur le plan technique, le capteur Grove fonctionne de manière analogique : il génère une tension électrique variable proportionnelle à la conductivité du milieu (ici, l'eau). Le microcontrôleur ne pouvant traiter que des informations binaires, il utilise son convertisseur analogique-numérique (ADC) pour transformer cette tension en un nombre entier. C'est ce nombre, image fidèle de la tension mesurée au multimètre, qui est ensuite utilisé par notre programme pour piloter l'arrosage.

Interprétation de résultat : Appliquant la règle du trois 3V3 ==> 4095

1V45 ==> 1745

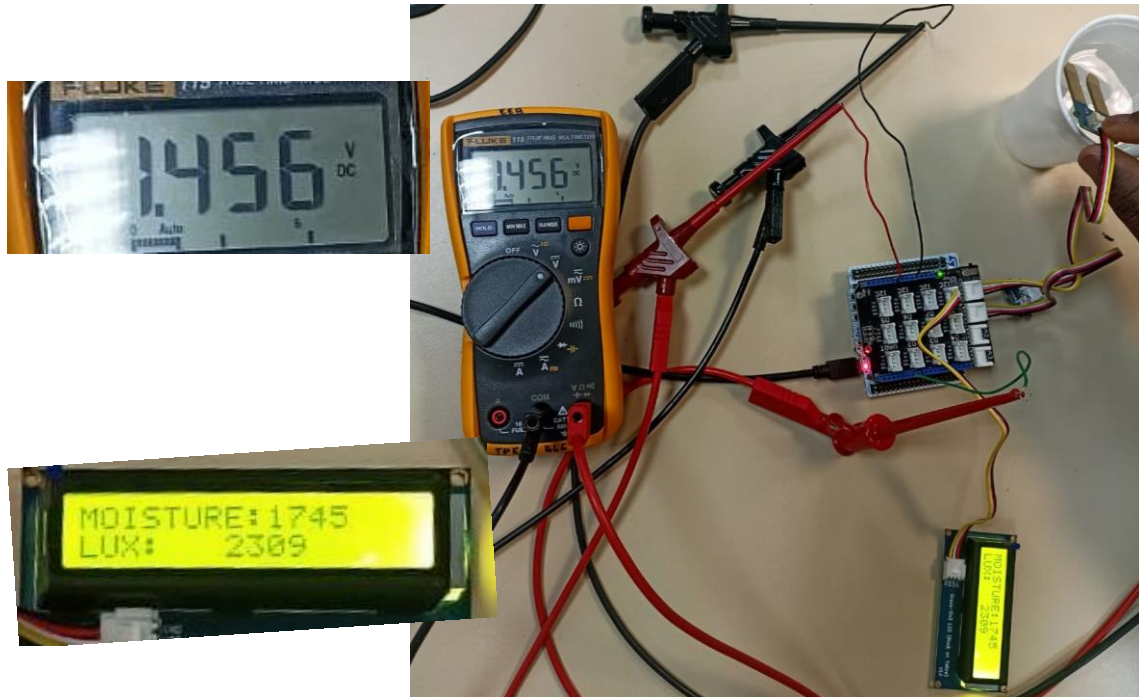


Figure : Affichage de données de capteurs humidité et luminosité sur LCD

Manipulation 4 : Programmation et test du MOSFET

Et voici le code rajouté dans le main.c concernant la gestion du signal MLI (PWM) :

```
// Dans la section des initialisations (au début du main)
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);

// Variables pour la temporisation de l'arrosage
static uint32_t last_arrosage = 0;

// Dans la boucle while(1)
// Sortie PWM pour la commande
// Sortie sur PA8
if (get_command()){
    __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 200); //20% de la tension
}
else{
    if ( HAL_GetTick()-last_arrosage > 1000 ){
        //Lorsque le sol est très sec
        // On alimente à 40% de la tension
        if ( moisture >= 0 && moisture < 300 )
            __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 400); // 40% de la tension

        // Lorsque le sol est un peu humide
        // On alimente à 20% de la tension
        if ( moisture >= 300 && moisture < 700 )
```

```

    __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 200);
    // Lorsque le sol est humide
    // On cesse d'alimenter le moteur
    if( moisture >=700)
        __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 0);

    last_arrosage = HAL_GetTick();
}
}

```

Manipulation 5 : Test de l'ensemble de composants

Voici le système finalisé en condition de test. Ces images mettent en avant le montage matériel (câblage) ainsi que la validation des signaux PWM via l'oscilloscope.

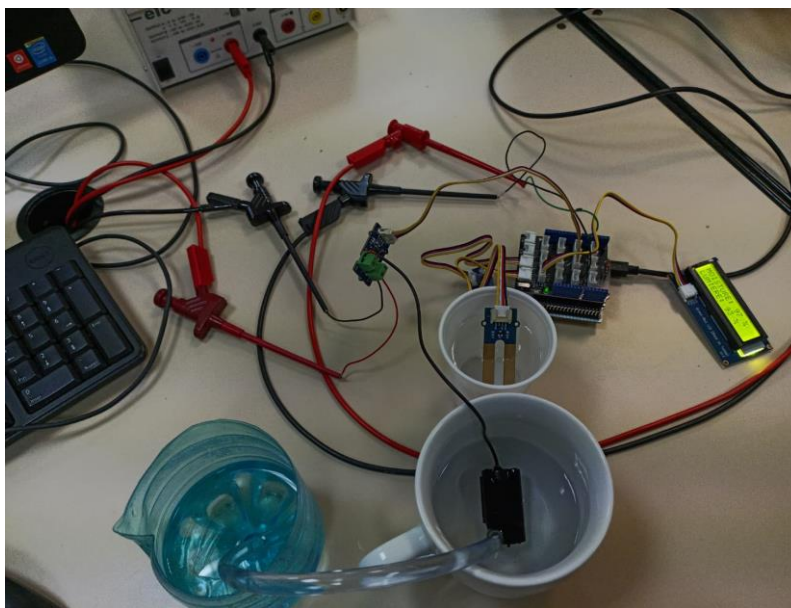


Figure 9: Affichage de données de capteurs humidité et luminosité sur LCD

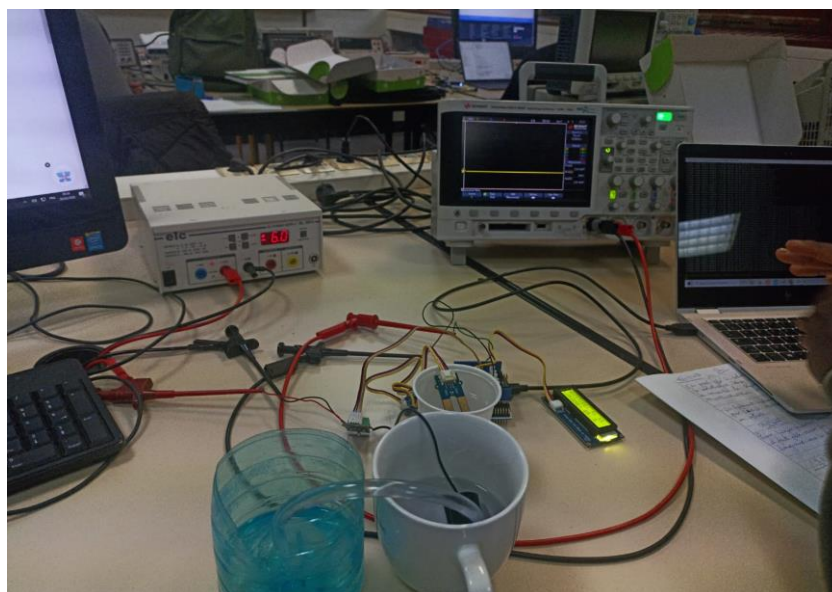


Figure 10 : Affichage de données de capteurs humidité et luminosité sur LCD

Manipulation 6 : Commande du système par téléphone (Bluetooth)

Les figures suivantes montrent les différentes étapes qu'on suit à chaque fois pour se connecter afin qu'on envoie une commande pour forcer l'arrosage à distance et qu'on reçoit des données des capteurs d'humidité et de lumière.

Le bouton Write indique l'envoi de la commande, et les mots affichés en dessous de UNKNOWN indiquent les valeurs mesurées des capteurs en temps réel, mais elles sont exprimées en ASCII.

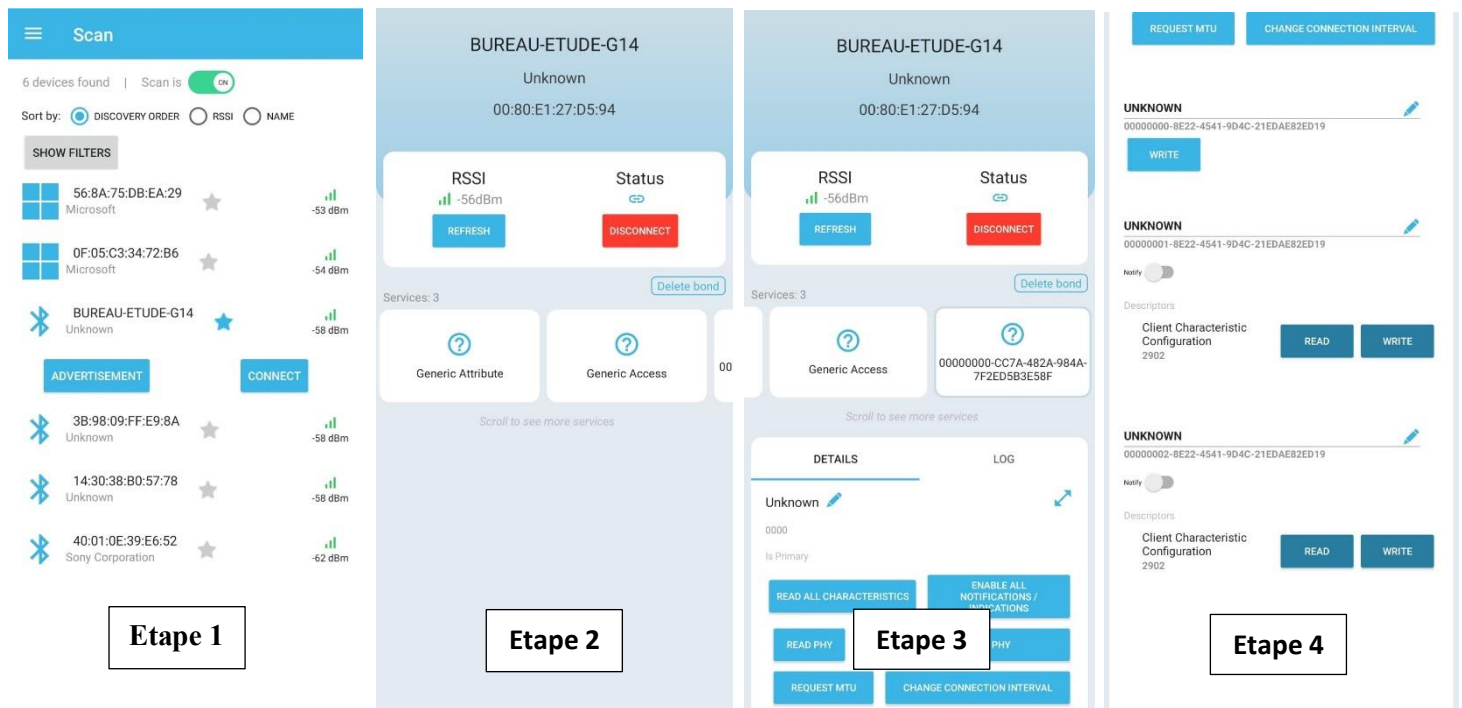


Figure 11: Captures d'écran de l'application Bluetooth

C'est pour cela on a pensé à convertir ces valeurs mesurées en ASCII en graphe pour mieux comprendre ces mesures et que pour s'assurer que les valeurs sont similaires à ceux affichées sur le LCD.

Et voici le code rajouté dans le main.c pour commander le système à travers l'application du ST BLE ToolBox :

```
// Dans la boucle while(1)
if (get_command()){
    __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, 200); //20% de la tension
}
else {
    // Asservissement normal
}

// Mise à jour des valeurs dans la stack Bluetooth
set_lux(lux);
set_mois(moisture);
```

Cette figure illustre le signal PWM de notre système observé à l'oscilloscope. On y distingue deux configurations du rapport cyclique, respectivement à 10 % et 20 %.



Figure 12: Rapport cyclique défini dans le code 10% et 20%

Manipulation 7 : Rendre le système autonome (alimentation externe)

D'après la datasheet de la carte nucleo-wb55rg, on a su que la carte s'alimente avec une tension qui varie entre 3V3 et 5V. En partant de 3,7 V, même avec une petite chute, on reste confortablement au-dessus de 3,3 V. Et cette même alimentation est partagée avec la partie puissance et la partie commande (carte + Mosfet et pompe).

Pour cette alimentation stabilisée de 3V7 reçue du générateur DC, on a placé un ampèremètre pour mesurer le courant consommé.

Donc on a déduit que la consommation restait stable que se soit en repos, en fonctionnement (arrosage) ou en envoi/réception par Bluetooth, et c'est égal à 57.9mA comme indiqué sur le multimètre.

Pour savoir la puissance, c'est le produit du courant et la tension, et qui vaut $3.7 \text{ V} * 57.9 \text{ mA} = 214 \text{ mW}$.

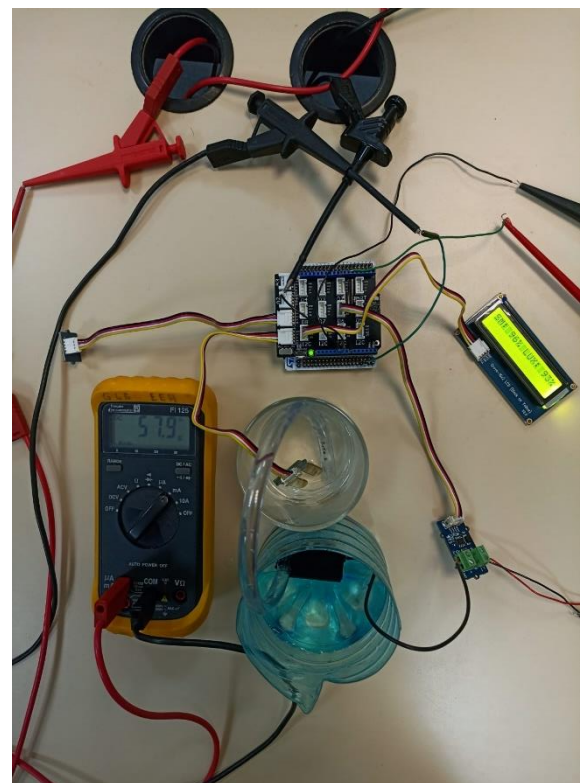


Figure 13: Système Complet testé en autonomie

Concernant l'interconnexion de notre plante connectée avec la batterie (dans notre cas on l'a remplacé par un générateur DC), on a connecté le V+ de l'alimentation avec la broche Vbat (broche 33) selon la figure ci-dessous :



Figure 14 : Brochage de Vbat [3]

Finalement, nous avons conclu notre travail par l'analyse du spectre des signaux échangés entre le BLE du téléphone et celui de la carte Nucleo en utilisant l'analyseur de spectre, qui est un appareil de mesure qui permet de visualiser les signaux radiofréquences. Il montre la puissance du signal en fonction de la fréquence et permet de vérifier le bon fonctionnement de l'émetteur.

Cet essai réalisé avec l'analyseur de spectre a permis de mesurer une fréquence de 2,45 GHz, validant ainsi le bon fonctionnement de l'émetteur BLE de la carte Nucleo WB55RG, dont la plage de fonctionnement nominale se situe entre 2,405 GHz et 2,480 GHz.



Remarque : les puissances de signaux (mesuré et celle indiquée par le BLE du téléphone) sont égales.

Figure 15 : Analyseur de spectre de la fréquence BLE

Bibliographie :

- [1] https://fr.vittascience.com/public/content/user_data/resources/Guide_utilisation_plante.pdf
- [2] https://stm32python.gitlab.io/fr/docs/Kit/nucleo_wb55rg
- [3] <https://os.mbed.com/platforms/ST-Nucleo-WB55RG/>
- [4] https://wiki.seeedstudio.com/Grove-Moisture_Sensor/
- [5] https://wiki.seeedstudio.com/Grove-Light_Sensor/